LECTURE 15

Agents

The combination of LLMs and genetic algorithms we saw in the previous lecture is just one example of a broader class of systems that incorporate LLMs into larger workflows that include other code and algorithms in order to solve a problems. In recent years, there has been a lot of interest in building such systems, often called agents.

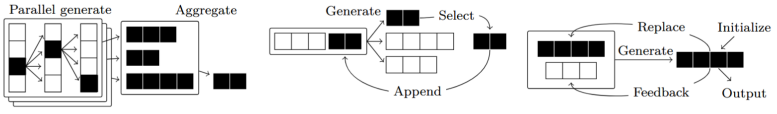
We can begin a very broad classification of agents into two categories: **Program-in-control** and **LLM-in-control**. A program-in-control agent is basically a python program that invokes an LLM to help it solve the problem. There is a broad range of such agents, depending on how much of the problem solving is done by the LLM versus by the program. At one end of the spectrum, we have search-based agents, where the program implements a generic search strategy, but where all the problem specific logic is done by the LLM. At the other end of the spectrum, we have domain-specific workflows, where the program implements a workflow that is specific to the problem at hand, and where the LLM is just invoked to perform sub-tasks that cannot be performed with traditional code.

An LLM-in-control agent is basically an LLM that is augmented with the ability to invoke code to help it solve problems. This is often referred to as tool use, but it can be more general than that. In some cases, the LLM is not limited to the use of existing tools, but can also generate code to solve a sub-task that is better suited to algorithmic solution. In the rest of this lecture, we elaborate on these different kinds of agents.

15.1 Search-based agents

Generic agent workflows

From Welleck et al. in TMLR 24; they describe these as meta-generation algorithms



(a) Parallel search (b) Heuristic step-level search (c) Refinement

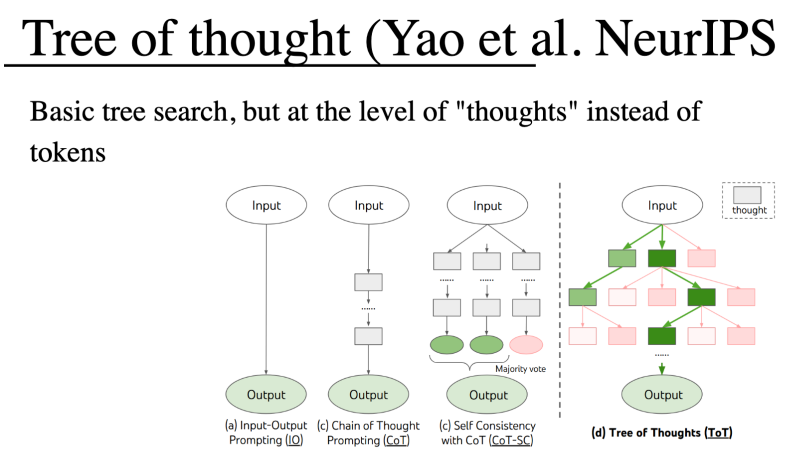


Figure 1 from the Yao et al. paper

In many situations, the problem we want to solve can in principle be solved by an LLM, but the LLM is not reliable enough to ensure that the solution it provides is correct. In such situations, it is common to use code to orchestrate repeated invokations of the LLM in order to search for a correct solution.

Welleck et al [1] call these meta-generation algorithms, and they provide a good survey of them. Snell et al. [2] also explore several such algorithms and argue that scaling test time compute can be more effective than scaling model size for improving performance on reasoning tasks.

The figure illustrates some of the meta-generation algorithms described by Welleck et al. Parallel search involves invoking the LLM multiple times in parallel to generate candidate solutions, and then evaluating them aggregating their solutions to produce a final answer. Heuristic step-level search involves generating many partial solutions, and then selecting among them to chose next step of the solution. Refinement involves generating tentative solutions and then asking a model to critique them and then using the last solution and the generated feedback to ask the model to improve the solution.

In NeurIPS 2023, Yao et al [3] introduced an algorithm that has become fairly popular called Tree of Thoughts, which performs a tree search over the outputs of an LLM. The idea is to perform a tree search over the outputs of the model, but instead of doing it at the granularity of tokens, to do it at the granularity of "thoughts", which are coherent units of text that represent one step in a reasoning process.

15.2 Domain specific workflows

LMQL — Examples from the Beurer-Kellner et al. 2023 paper

Query

```
beam(n=3)
  "A list of good dad jokes. A indicates the "
  "punchline \n"
  "Q: How does a penguin build its house? \n"
  "A: Igloos it together. END \n"
  "Q: Which knight invented King Arthur's Round"
  "Table? \n"
  "A: Sir Cumference. END \n"
  "Q: [JOKE] \n"
  "A: [PUNCHLINE] \n"
from "gpt2-medium"
where
  STOPS_AT(JOKE, "?") and STOPS_AT(PUNCHLINE, "END")
  and len(words(JOKE)) < 20
  and len(characters(PUNCHLINE)) > 10
```

The actual output of the LLM will be written into the variables JOKE and PUNCHLINE

Constraints

(a) LMQL query to generate a joke.

LMQL — Examples from the Beurer-Kellner et al. 2023 paper

Query

```
beam(n=3)
  "A list of good dad jokes. A indicates the "
  "punchline \n"
  "Q: How does a penguin build its house? \n"
  "A: Igloos it together. END \n"
  "Q: Which knight invented King Arthur's Round"
  "Table? \n"
  "A: Sir Cumference. END \n"
  "Q: [JOKE] \n"
  "A: [PUNCHLINE] \n"
from "gpt2-medium"
where
  STOPS_AT(JOKE, "?") and STOPS_AT(PUNCHLINE, "END")
  and len(words(JOKE)) < 20
  and len(characters(PUNCHLINE)) > 10
```

The constraints describe where JOKE and PUNCHLINE end, as well as additional syntactic requirements, for example on their length

Constraints

(a) LMQL query to generate a joke.

LMQL — Examples from the Beurer-Kellner et al. 2023 paper

This specifies that the model is expected to generate 2 THINGs that will be added to the things list as they are generated.

Things are constrained to belong to a predefined list of choices. The runtime enforces this through constrained decoding

```
argmax
  "A list of things not to forget when "
  "travelling:\n"
  things = []
  for i in range(2):
    "- [THING]\n"
    things.append(THING)
  "The most important of these is [ITEM]."
from "EleutherAI/gpt-j-6B"
where
   THING in ["passport",
             "phone",
             "keys", ...] // a longer list
   and len(words(THING)) <= 2
```

(b) LMQL query utilizing a python list.

There are many situations where we want to build more specialized workflows that are tailored to a specific problem. There has been some recent work on building languages and frameworks for specifying such workflows. A particularly notable one from the programming languages community is the LMQL language [4]. The basic idea in this language is to provide simple syntax to support the user to describe workflows where information provided by an LLM in one step is used to construct the prompt for the next step.

The figure shows some examples that showcase the use of LMQL (from the LMQL paper) [4]. The basic idea of LMQL is that you can write a prompt template that uses variables in brackets to indicate places where information will be filled in by the LLM, and then you can use those variables in later parts of the prompt. Interspersed with the prompt, you can use Python code to process the outputs of the LLM and to compute values that will be used in later parts of the prompt.

The language also allows the user to specify constraints on the outputs of the LLM, and the LMQL runtime will ensure that the outputs satisfy those constraints.

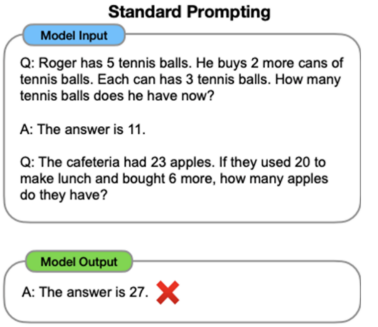
15.3 LLM-in-control style agents

Logic problems

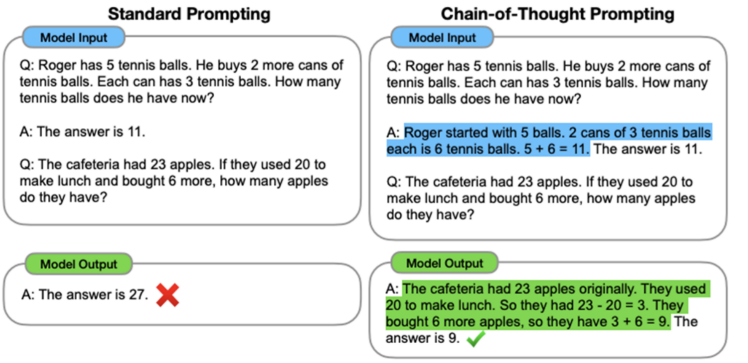
Input

```
Premise:     All rectangles have four sides.
Premise:     All four-sided things are shapes.
Conclusion:  Are all rectangles shapes?
```

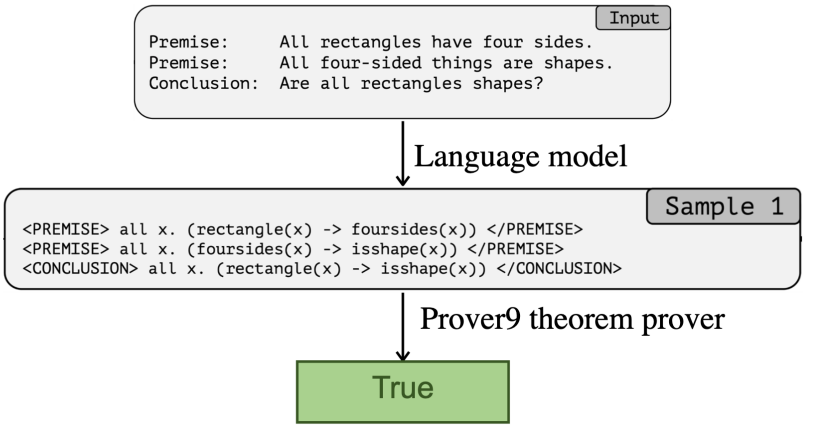
Chain of Thought (COT)



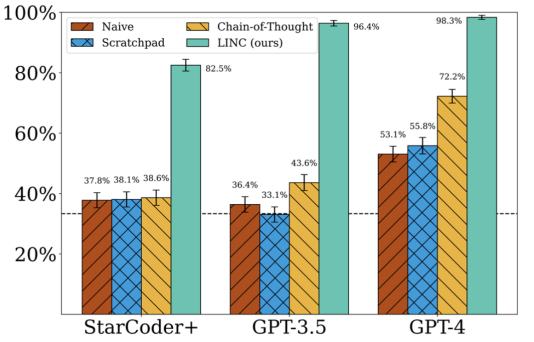
Chain of Thought (COT)



The LINC approach

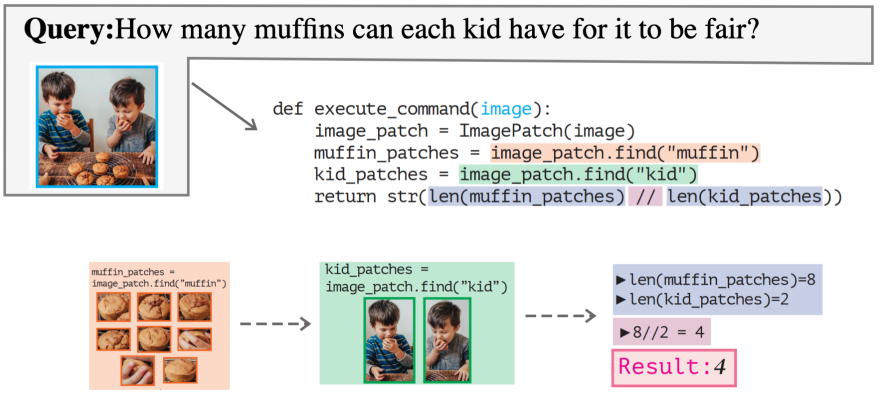


ProofWriter Benchmark Results



ViperGPT

(Example from the ViperGPT paper)



In contrast to the program-in-control agents we have seen so far, there has been a lot of recent interest in LLM-in-control agents, where the LLM is the main driver of the problem solving process, but where it is augmented with the ability to invoke code. A simple example of this is the tool-use paradigm, where the LLM is given access to a set of tools (APIs) that it can invoke to help it solve problems. One example of this is the LINC paper which explored the use of a theorem prover to solve logical problems that may otherwise be too difficult for the LLM to solve on its own [5]. In this case, the LLM is prompted to translate the problem into the input notation of the Prover 9 theorem prover so it can leverage the reasoning capabilities of the theorem prover.

This idea of leveraging code generation for problem solving was the basis of the ViperGPT project [6], which explored the idea of using an LLM to generate code to solve a problem instead of solving the problem directly. The focus of the paper was on visual reasoning tasks, and the LLM was given access to an API with a set of machine vision primitives and was prompted to generate code that uses those primitives to solve the visual reasoning task.

A similar idea was explored in the Chain of Code paper [7], which also generates code to solve problems rather than solving them directly. A big novelty in this paper is that the generated code may invoke functions that are not implemented in code. When the interpeter reaches one such function, it calls the LLM to emulate the function.

References

[1] Sean Welleck, Amanda Bertsch, Matthew Finlayson, Hailey Schoelkopf, Alex Xie, Graham Neubig, Ilia Kulikov, and Zaïd Harchaoui. From decoding to meta-generation: Inference-time algorithms for large language models. *Trans. Mach. Learn. Res.*, 2024, 2024. URL https://openreview.net/forum?id=eskQMcIbMS.

[2] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR) 2025*, 2025. URL https://openreview.net/pdf?id=4FWAwZtd2n.

[3] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS) 2023*, 2023. URL https://arxiv.org/abs/2305.10601.

[4] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, 2023. URL https://dl.acm.org/doi/pdf/10.1145/3591300.

[5] Theo Olausson, Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 5153–5176. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.EMNLP-MAIN.313. URL https://doi.org/10.18653/v1/2023.emnlp-main.313.

[6] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) 2023*, 2023. URL https://arxiv.org/abs/2303.08128.

[7] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. Chain of code: Reasoning with a language model-augmented code emulator. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*,

volume 235 of *Proceedings of Machine Learning Research*, pages 28259–28277. PMLR, 2024. URL https://proceedings.mlr.press/v235/li24ar.html.